

Document number: P0889R0
Project: Programming Language C++
Audience: Evolution Working group

Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2018-01-08

Ultimate copy elision

I. Quick Introduction

[`class.copy.elision`] in the current working draft [[N4713](#)] provides a whitelist of cases when copy elision is permitted. Those rules are good! However they do not take into account that modern compilers could inline functions and do other optimizations.

This paper motivates and proposes to relax the [`class.copy.elision`] rules in order to allow compilers to produce much better code.

II. The pitfall

A. Teaching practice

For decades, almost all teaching materials were telling people to decompose their programs into functions for maintainability and code re-usability. That good advice leads to code with numerous functions. Compiler developers noted that and started inlining functions more aggressively.

Currently inlining is one of the major optimizations [[Understanding Compiler Optimization, 26m20s by Chandler Carruth](#)] and compilers inline a lot.

B. Copy elision rules

Current rules for copy elision mostly assume that a function from source code remains a function in a binary. This works perfectly fine in a world without inlining, aliasing reasoning, and link time optimization. But if the function is inlined, then the compiler "sees" the whole function body along with function parameters. This unleashes a whole new world of possible optimizations (See section III for examples), but existing copy elision rules prevent those optimizations.

Our current rules are suboptimal for modern compilers: they prevent optimizations.

C. `std::move/rvalues` is not a panacea

We have `std::array`, `std::basic_string`, `std::function`, `std::variant`, `std::optional` and other classes that may store a lot of data on the stack. "Moving" instances of those classes may result in copying a lot of bytes.

Copy elision could be more profitable.

D. `std::move/rvalues` rules may be obscure

What is the optimal way to return something from a function? For named object we must just return. For a subobject we must return it with `std::move`. For a function parameter we must return it by `std::move`. If we return a reference to a local object, we have to `std::move` it. We must also return elements from structured binding via `std::move`, but only if the element is not a reference to an value returned by reference before decomposition... Do we have to apply `std::move` for a member function call on a local variable when the function call returns reference?..

Beginners and even some advanced programmers do not always know about those obscure rules and already assume that the compiler will do the job for them.

E. In sum

We have for decades been teaching people to write functions. WG21 was improving the C++ language by providing features for advanced programmers to optimize the functions, leaving behind the abilities of modern compilers to do that job sometimes better and sometimes out-of-the-box.

III. Examples of possible optimizations

All the examples in this section use the following classes and assume that `callee` is inlined by the optimizer:

```
struct detect_copy {
    detect_copy() noexcept = default;
    detect_copy(const detect_copy&) noexcept;
    ~detect_copy();
    int modify() noexcept;
};

struct pair {
    detect_copy first, second;
};
```

A. Returning a function parameter

```
static detect_copy callee(detect_copy v) {
    return v;
}

int caller() {
    auto v = callee(detect_copy{});
    return v.modify();
}
```

[[class.copy.elision](#)] forbids copy elision if a function parameter is returned. However, modern compilers do inline the callee. This results in a copy constructor call immediately followed by a call to the destructor: <https://godbolt.org/g/nYovU3>.

Code could be optimized by the compiler to the following, avoiding calls to the copy constructor and destructor:

```
int caller() {
    detect_copy v;
    return v.modify();
}
```

B. Returning a reference to a local

```
static detect_copy callee() {  
    detect_copy v;  
    auto& ref = v;  
    return ref;  
}  
  
int caller() {  
    return callee().modify();  
}
```

[class.copy.elision] forbids copy elision if a reference is returned. However, modern compilers do understand that ref is just a reference to v. This can be seen from the assembly, where no separate variable/register is used for a ref: <https://godbolt.org/g/YMAAN4>. Note the call to the copy constructor immediately followed by a call to the destructor.

It means that the code could be optimized by the compiler to avoid calls to the copy constructor and destructor:

```
int caller() {  
    detect_copy v;  
    return v.modify();  
}
```

C. Returning a subobject

```
static detect_copy callee() {  
    pair v;  
    return v.second;  
}  
  
int caller() {  
    return callee().modify();  
}
```

[class.copy.elision] forbids copy elision if a subobject is returned. However modern compilers do understand that pair could be treated as two detect_copy variables because pair has a default destructor. The copy constructor call is immediately followed by a call to the destructor for the same register: <https://godbolt.org/g/kyPR7R>.

Code could be optimized by the compiler to avoid calls to the copy constructor and destructor:

```
int caller() {  
    detect_copy first;  
    { detect_copy second; }  
    return first.modify();  
}
```

D. Returning a union element

```
union optional {  
    bool fake;  
    detect_copy data;  
  
    optional()  
}
```

```

        : data{}
    {}

    ~optional(){
        data.~detect_copy();
    }
};

static detect_copy callee() {
    optional v;
    return v.data;
}

int caller() {
    return callee().modify();
}

```

[class.copy.elision] forbids copy elision if a union element is returned. However modern compilers have knowledge of the active union member, because they do check that in constexpr calls. In the above example, the copy constructor call is immediately followed by a call to the destructor for the same memory:

<https://godbolt.org/g/Udb7vN>.

Code could be optimized by the compiler to avoid calls to the copy constructor and destructor:

```

int caller() {
    detect_copy v;
    return v.modify();
}

```

E. Returning an element from a structured binding

```

static detect_copy callee() {
    auto [f, s] = pair{};
    return s;
}

int caller() {
    return callee().modify();
}

```

[class.copy.elision] forbids copy elision if a reference to the subobject is returned. However, modern compilers do understand that `s` is just a reference to `pair{} .second`. This can be seen from the assembly, where no separate variable/register is used for a `s`: <https://godbolt.org/g/quV9Cp>. Note the call to the copy constructor immediately followed by a call to the destructor for the same memory.

Code could be optimized by the compiler to avoid calls to the copy constructor and destructor:

```

int caller() {
    detect_copy first;
    { detect_copy second; }
    return first.modify();
}

```

F. Returning a local variable that is `std::moved` in `return`

```

static detect_copy callee() {
    detect_copy v;

```

```

        return static_cast<detect_copy&&>(v);
    }

    int caller() {
        auto v = callee();
        return v.modify();
    }

```

[class.copy.elision] forbids copy elision if an rvalue reference is returned. However modern compilers do understand that actually `v` is returned. This can be seen from the assembly, where the compiler operates with just an address of `v` and does not use separate variables/registers for a reference: <https://godbolt.org/g/bPQ8Ja>. Note the call to the copy constructor immediately followed by a call to the destructor.

It means that the code could be optimized by the compiler to avoid calls to the copy constructor and destructor:

```

    int caller() {
        detect_copy v;
        return v.modify();
    }

```

G. Passing variable that is not used any more by copy

```

    int takes_by_copy(detect_copy /*v*/) noexcept {}

    int caller() {
        detect_copy v;
        return takes_by_copy(v);
    }

```

[class.copy.elision] forbids copy elision in that case. Thou it is obvious that `v` is not used any more, it can not be accessed from `takes_by_copy` function. The assembly is suboptimal because of unnecessary copy construction and destruction: <https://godbolt.org/g/F7vTCM>. Note the call to the copy constructor immediately followed by a call to the destructor.

Code could be optimized by the compiler to avoid calls to the copy constructor and destructor. In that case even inlining of the `takes_by_copy` function is not required:

```

    int takes_by_copy(detect_copy /*v*/) noexcept {}

    int caller() {
        return takes_by_copy({});
    }

```

H. Returning a reference to a subobject of a local object

```

class stringstream {
    detect_copy internal;

public:
    detect_copy& str() { return internal; }
};

static detect_copy callee() {
    stringstream ss;

```

```

        return ss.str();
    }

    int caller() {
        return callee().modify();
    }

```

[class.copy.elision] forbids copy elision in that case. Thou the compilers succeeded in understanding and inlining the `stringstream::str()` function: <https://godbolt.org/g/MjEocB>. Note the call to the copy constructor immediately followed by a call to the destructor.

Code could be optimized by the compiler to avoid calls to the copy constructor and destructor:

```

    int caller() {
        return detect_copy{}.modify();
    }

```

I. Storing a parameter in a class

```

struct B {
    detect_copy a;
    B(const detect_copy& a): a(a) { }
};

int main() {
    (B(detect_copy()));
}

```

[class.copy.elision] forbids copy elision in that case. This issue was reported in [CWG #1049](#).

In the disassembly, we see a call to the copy constructor is immediately followed by a call to the destructor: <https://godbolt.org/g/zFWJNc>.

IV. General idea

All the examples from above were producing code that after inlining (and some other optimizations) contains copy/move constructor call followed by a call to destructor. Something close to the following could be found in disassembly:

<code>caller():</code>	<code>; int caller() {</code>
<code>sub rsp, 24</code>	<code>;</code>
<code>lea rdi, [rsp+13]</code>	<code>;</code>
<code>call A::A()</code>	<code> A a;</code>
<code><...></code>	<code> // some other code</code>
<code>lea rsi, [rsp+13]</code>	<code>;</code>
<code>lea rdi, [rsp+14]</code>	<code>;</code>
<code>call A::A(A const&)</code>	<code> A b = a;</code>
<code>lea rdi, [rsp+13]</code>	<code>;</code>
<code>call A::~~A()</code>	<code> a.~A();</code>
<code><...></code>	<code> // some other code</code>
<code>lea rsi, [rsp+14]</code>	<code>;</code>
<code>lea rdi, [rsp+15]</code>	<code>;</code>
<code>call A::A(A&&)</code>	<code> A c = std::move(b);</code>
<code>lea rdi, [rsp+14]</code>	<code>;</code>
<code>call A::~~A()</code>	<code> b.~A();</code>
<code><...></code>	<code> // some other code</code>

```

    lea    rdi, [rsp+15]    ;
    call   A::~A()          ;    c.~A();
    <...>    ;              ;    // some other code
                ; }

```

The idea of this paper is to allow the compiler to elide those copies:

```

caller():                                ; int caller() {
    sub    rsp, 24                    ;
    lea    rdi, [rsp+15]              ;
    call   A::A()                     ;    A a;
    <...>    ;                          ;    // some other code
    <...>    ;                          ;    // some other code
    <...>    ;                          ;    // some other code
    lea    rdi, [rsp+15]              ;
    call   A::~A()                     ;    a.~A(); // previously c.~A();
    <...>    ;                          ;    // some other code
                ; }

```

V. Addressing common concerns

Concern: Eliding copy/move constructors could break user code

Response: Yes, but only if the following generally-accepted constraint for a copy constructor is not satisfied: "After the definition $\tau \ u = v;$, u is equal to v ".

Although the constraint seems very restrictive at first, that constraint is satisfied by every sane copy constructor. Moreover the C++ Language and Standard Library heavily rely on it:

- The Ranges TS forces the requirement that objects after copy/assignment/move are equal (N4651: "After the definition $T \ u = v;$, u is equal to v " in Concept CopyConstructible. The same rules can be found in Concept Assignable and Concept Swappable).
- The standard library implicitly requires objects after copy/assignment/move to be equal in [container.requirements.general].
- Algorithms do not work well if that constraint is not satisfied, as some of the complexities in [alg*] are described as "At most $.*$ swaps" or "Approximately $.*$ swaps". Moreover, algorithms sometimes do not specify the order of copying/swapping so if the requirement is not satisfied the results could be different on different platforms.
- [class.copy.elision] implicitly relies on that constraint.
- WG21 implicitly relied on that constraint in "P0135R1: Wording for guaranteed copy elision through simplified value categories".

In other words, WG21 has been relying on that constraint for a long time and classes that violate that constraint are already unportable.

Concern: The examples do not look like a real world code. Nobody writes such bad code

Response: Examples are simplified just to show that the optimization is possible.

Real code would have functions located in different headers, more code will be in the function body. We searched the Yandex code base for `return.*first;` and `return.*second;` and found thousands of matches. Note that we searched only for a single optimization case for only a `std::pair`. Tuples, aggregates and more complex data types are also affected.

Concern: Advanced optimizations could affect compilation time

That depends. Making a high level optimization could be faster than doing a lot of low level optimizations. For example removing the copy constructor and destructor could be faster than inlining both and optimizing all the internals.

Anyway, to implement or not to implement a particular optimization is up to the compiler developers. This proposal just attempts to untie their hands and allow more optimizations.

Concern: It is impossible to implement some of the optimizations right now.

Response: This proposal does not require any of the optimizations from examples. The proposal simply attempts to relax copy elision rules to allow those optimizations someday.

Concern: We want an emergency hatch to disable copy elision for particular places.

Response: Our usual practice to disable optimizations for a variable is to make it `volatile`. This feature must be kept.

Concern: The optimizations are not guaranteed, so users still have to write `std::move`

Response: That's true. Proposals for automatically applying `std::move` may be a good idea. Such proposals would be more restrictive than the copy elision rules because we have no control over all the compiler inlining and reasoning logic. We can not guarantee that all the compilers would be able to inline some function or would be able to understand that the reference references a local object.

Moving in both directions would produce better results:

- simplifying copy elision rules will remove border cases when `std::move` hurts performance
 - implicit casting to rvalues will simplify the code and allow writing less `std::move`s
 - implicit casting to rvalues and more generic copy elision rules will produce faster programs out-of-the-box. Users will have to worry less for performance. This will simplify the language usage and teaching materials (See "D. `std::move`/rvalues rules may be obscure").
-

Concern: How this would change the guidelines? How users shall write their code to get the benefits of this optimization?

Response: This optimization won't change the guidelines in the nearest future. Treat this optimization as one of the compiler optimizations that you could not directly control, like inlining, jump threading, loop fusion, or common subexpression elimination.

Some guidelines may change after major compilers adopt the optimization. In that case, there could be found a common pattern that triggers it and that pattern could be taught. Probably the existing guidelines some day would evolve into "If you've got a function bigger than 15 lines of code, you may use `std::move` for returning objects that are not constructed at the beginning of the function. Otherwise just return the value."

Concern: Extracting a subobject from an object is scary. I can no longer assume that the object I construct as a subobject is in any way part of myself, nor can it assume that any sibling subobjects will always be around.

Response: The proposed wording makes sure that the subobject is not used between copy/move construction and destruction. An object that is constructed as a subobject is a part of the object as long as you use it as a subobject. As soon as you copy/move and destroy it, you can not use it any more by existing C++ rules. That's the place where the optimization steps in, removing the copy/move+destruction and reusing the subobject.

Concern: The problem is not that big because compilers could inline the constructors and destructors and then optimize the resulting code

Response: Yes they do that. But the resulting code is still suboptimal, because even for `std::string` compilers could not optimize away all the dead stores. Consider the following example:

```
#include <utility>
#include <string>

std::pair<std::string, std::string> produce();

static std::string first_non_empty_move() {
    auto v = produce();
    if (!v.first.empty()) {
        return std::move(v.first);
    }
    return std::move(v.second);
}

int example_1_move() {
    return first_non_empty_move().size();
}
```

Note that `std::move` is used and everything must be optimal. But it's not, because with this paper's proposed copy elision rules, the resulting code could still be much shorter and with fewer conditional jumps:

Without copy elision https://godbolt.org/g/3xZvtw	With copy elision https://godbolt.org/g/RiTmPE
<pre>example_1_move(): push rbp push rbx sub rsp, 104 lea rdi, [rsp+32] mov rbx, rsp call produce[abi:cxx11]() mov rax, QWORD PTR [rsp+40] test rax, rax je .L2 lea rdx, [rbx+16] lea rcx, [rsp+48] mov QWORD PTR [rsp], rdx mov rdx, QWORD PTR [rsp+32] cmp rdx, rcx je .L13 mov QWORD PTR [rsp], rdx mov rdx, QWORD PTR [rsp+48] mov QWORD PTR [rsp+16], rdx .L4: mov QWORD PTR [rsp+8], rax lea rax, [rsp+48] mov rdi, QWORD PTR [rsp+64]</pre>	<pre>example_1_optimized(): push rbx sub rsp, 64 mov rdi, rsp call produce[abi:cxx11]() mov rax, QWORD PTR [rsp+8] test rax, rax mov ebx, eax jne .L3 mov ebx, DWORD PTR [rsp+40] .L3: mov rdi, QWORD PTR [rsp+32] lea rax, [rsp+48] cmp rdi, rax je .L4 call operator delete(void*) .L4: mov rdi, QWORD PTR [rsp] lea rdx, [rsp+16] cmp rdi, rdx je .L1 call operator delete(void*) .L1:</pre>

<pre> mov QWORD PTR [rsp+40], 0 mov BYTE PTR [rsp+48], 0 mov QWORD PTR [rsp+32], rax lea rax, [rsp+80] cmp rdi, rax je .L6 call operator delete(void*) jmp .L6 .L2: lea rax, [rbx+16] lea rdx, [rsp+80] mov QWORD PTR [rsp], rax mov rax, QWORD PTR [rsp+64] cmp rax, rdx je .L14 mov QWORD PTR [rsp], rax mov rax, QWORD PTR [rsp+80] mov QWORD PTR [rsp+16], rax .L8: mov rax, QWORD PTR [rsp+72] mov QWORD PTR [rsp+8], rax .L6: mov rdi, QWORD PTR [rsp+32] lea rax, [rsp+48] cmp rdi, rax je .L9 call operator delete(void*) .L9: mov rdi, QWORD PTR [rsp] add rbx, 16 mov rbp, QWORD PTR [rsp+8] cmp rdi, rbx je .L1 call operator delete(void*) .L1: add rsp, 104 mov eax, ebp pop rbx pop rbp ret .L14: movdqa xmm0, XMMWORD PTR [rsp+80] movaps XMMWORD PTR [rsp+16], xmm0 jmp .L8 .L13: movdqa xmm0, XMMWORD PTR [rsp+48] movaps XMMWORD PTR [rsp+16], xmm0 jmp .L4 </pre>	<pre> add rsp, 64 mov eax, ebx pop rbx ret </pre>
---	---

Inlining the constructors and destructors and optimization of the resulting code fails in many cases:

- It is much harder to analyze the resulting code because it's bigger, touches more memory and registers.
- Constructors and destructors may be located in a separate translation unit, making that inlinement possible only during link time optimization.
- Constrcutor or destructor could be too big for inlinement and in that case no optimization is possible.

VI. Proposed wording: Ultimate solution for relaxing [class.copy.elision]

Adjust the [class.copy.elision] paragraph 1 to allow copy elision of all objects and subobjects:

This elision of copy/move operations, called copy elision, is permitted in the following circumstances (which may be combined to eliminate multiple copies):

- in a return statement in a function with a class return type, when the expression is the name of a non-volatile automatic object (other than a function parameter or a variable introduced by the exception-declaration of a handler (18.3)) with the same type (ignoring cv-qualification) as the function return type, the copy/move operation can be omitted by constructing the automatic object directly into the function call's return object
- in a throw-expression (8.17), when the operand is the name of a non-volatile automatic object (other than a function or catch-clause parameter) whose scope does not extend beyond the end of the innermost enclosing try-block (if there is one), the copy/move operation from the operand to the exception object (18.1) can be omitted by constructing the automatic object directly into the exception object
- when the exception-declaration of an exception handler (Clause 18) declares an object of the same type (except for cv-qualification) as the exception object (18.1), the copy operation can be omitted by treating the exception-declaration as an alias for the exception object if the meaning of the program will be unchanged except for the execution of constructors and destructors for the object declared by the exception-declaration. [Note: There cannot be a move from the exception object because it is always an lvalue. – end note]

~~Copy elision is~~ Above copy elisions are required where an expression is evaluated in a context requiring a constant expression (8.6) and in constant initialization (6.8.3.2). [Note: Copy elision might not be performed if the same expression is evaluated in another context. – end note]

Additionally, copy elision is allowed for any non-volatile object with automatic storage duration and it's non-volatile nested objects if source is not accessed between a copy/move construction of it and its destruction.

VII. Other ways of relaxing [class.copy.elision]

We could deal with each of the elision cases from the Section III of this paper separately. Such an approach is used in our companion P0878 paper. But note that such an approach is not generic, consumes considerable time, and scales badly because it attempts to allow a specific optimization without a way to inspect the abilities of all the compilers. This increases a risk of spending a lot of time on a case that would not be

implementable in the nearest future or spending a lot of time on a case that is not profitable for that particular compiler.

It seems better to allow compiler developers to choose optimizations, as they are the ones who know the weak and strong parts of the underlying optimizer.

VIII. Acknowledgements

Many thanks to Walter E. Brown for fixing numerous issues in draft versions of this paper.

Many thanks to Marc Glisse for providing a reference to CWG #1049.

Many thanks to Nicol Bolas for raising many concerns.